

MY READING VILLAGE

Technical Demo: Flutter mobile
development,
architecture, stack, and how it all works

Fernando Pinto Villarroel – 2026



Contents



1. What is My Reading Village?
2. Mobile Development Fundamentals
3. Flutter & Dart
4. Technology Stack
5. Core Game Loop
6. Q&A



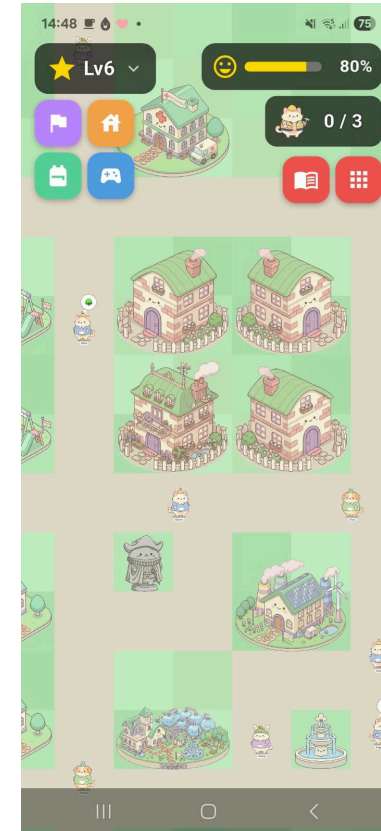
WHAT IS MY READING VILLAGE?

What is My Reading Village?

A **mobile app** that turns reading into a village-building game.

- ▶ Log pages read from any book
- ▶ Earn resources: coins, gems, wood, metal
- ▶ Spend them to construct buildings and grow a village
- ▶ Villagers move in as the village expands
- ▶ Missions, minigames, and a guided tour for new players

Goal: make reading a daily, rewarding habit through game mechanics, not a chore.





MOBILE DEVELOPMENT FUNDAMENTALS

Three kinds of software: three mental models



Backend API

Runs on a **server**. No UI. Receives HTTP requests, processes data, returns JSON or HTML.

Users never see this directly.

Node.js, Django, Spring, Go...

Web App

Runs **inside a browser**. HTML + CSS + JS render the UI. The browser is the host environment.

Must be fetched from a server each visit.

React, Vue, Angular, plain HTML...

Mobile App ← us

Runs **natively on device**. Installed once. Has direct access to hardware: camera, GPS, storage, notifications.

No browser. The OS is the host.

Flutter, React Native, Swift, Kotlin...

My Reading Village is a mobile app: it runs offline, persists data on-device, and renders a real-time game. No server, no browser, just Flutter on your phone.

Web app vs mobile app: concrete differences



	Web App	Mobile App (Flutter)
Distribution	URL, browser renders it	Installed package (.apk / .ipa), runs from device storage
Rendering	Browser engine (Blink, WebKit) paints DOM / CSS	Flutter's own rendering canvas (no browser dependency)
Offline	Partial, needs service workers and extra setup	First-class, app works with zero connectivity
Hardware	Limited, sandboxed by browser security model	Full access: camera, file system, notifications, sensors
Updates	Instant, users always get latest when opening URL	Must push a new release through App Store / Play Store



FLUTTER & DART

Flutter: one codebase, any device



Flutter is Google's open-source UI toolkit. Write code **once** in Dart, compile to native for Android, iOS, Web, and Desktop.

How is that possible?

Flutter ships its own **rendering engine**. It does not rely on each platform's native UI components, it draws every pixel itself, directly to the GPU canvas.

No browser. No bridge layer. Flutter compiles to native ARM machine code. Performance is comparable to apps written natively in Swift (iOS) or Kotlin (Android).

Everything in Flutter is a Widget

App Screen

Scaffold (page structure)

AppBar (navigation)

Body (content area)

Image (Flutter canvas)

HUD Overlay (buttons, bars)

Only the subtrees that changed are repainted, not the whole screen.

Dart: the language behind Flutter



Dart is Google's strongly-typed, object-oriented language, designed for client-side apps.

- ▶ **Statically typed:** errors caught at compile time, not runtime
- ▶ **Null safety:** the compiler prevents null-pointer crashes by design
- ▶ **Async-first:** concurrency built into the language itself
- ▶ **Single-threaded with isolates:** no shared-memory race conditions

Type System

Every value has a declared type. Mistakes are caught before the app runs, not when a user taps a button.

Null Safety

Variables cannot be null unless explicitly allowed. Eliminates an entire class of crashes.

Async / Await

Loading from disk or network never freezes the UI, async operations are a first-class language feature.

Familiar Syntax

Classes, interfaces, generics, lambdas, the same constructs you already know from Java or C#.



TECHNOLOGY STACK

Full tech stack at a glance



UI & Framework

Flutter 3.5+
Dart language
Material Design widgets
Custom kawaii-pastel theme

Game Engine

Flame 1.21
2D isometric grid
Sprite components
Camera & input system

Data & State

sqflite (SQLite)
13 tables, local DB
Provider (state mgmt)
GetIt (DI container)

Platform APIs

Push notifications
Camera & gallery
OS share sheet
File system access

Localization: 5 translation files (EN, ES, FR, IT, PT)

Networking: fetches book metadata from Open Library API

Extras: confetti animations · gallery export · backup & restore

Clean Architecture in Flutter



The codebase is split into **four layers**. Each layer has one responsibility. The key rule: **dependencies only point inward**, outer layers know about inner ones, never the reverse.

Why this matters: the database can be swapped for cloud storage, or the state library replaced, without touching any business logic. The innermost layer has zero framework imports.

This is the same “ports and adapters” pattern used in well-structured backend services, the same principles apply equally well on mobile.

Infrastructure (Flutter UI + Database) screens, widgets, game canvas, SQLite helper

↓ depends on

Adapters (Providers & Repositories) connects Flutter state to business services

↓ depends on

Application (Services) ReadingService, BuildingService, MissionService...

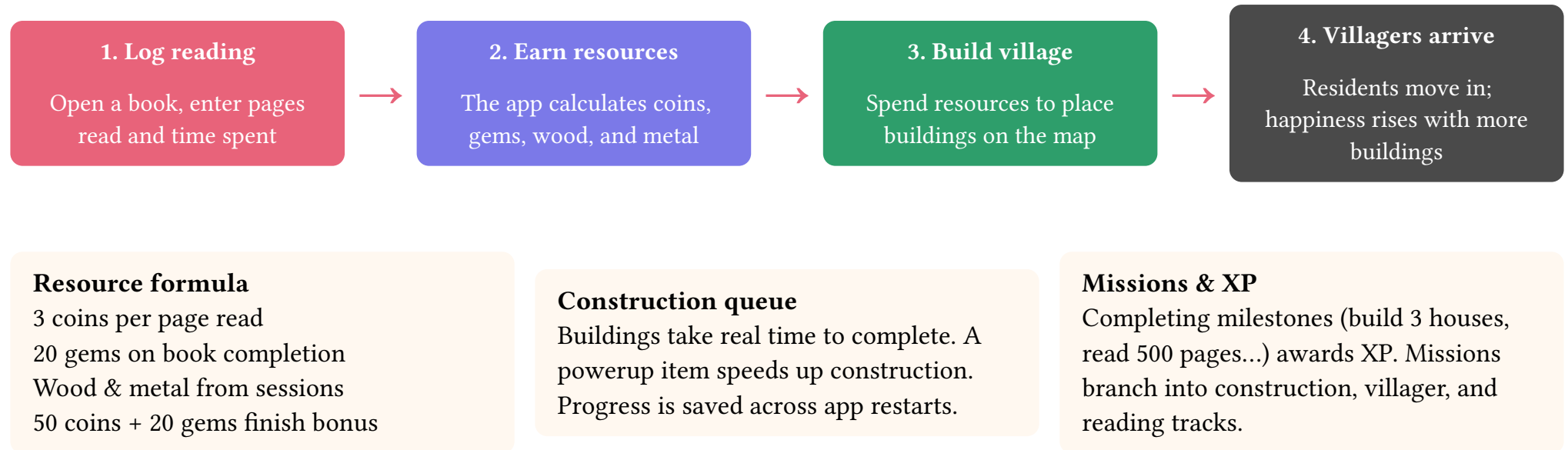
↓ depends on

Domain (Entities + Rules) Book, Villager, Building, pure Dart, no Flutter



CORE GAME LOOP

Core game loop: how reading becomes gameplay



Q&A



Thank you



My Reading Village: tech summary

Flutter 3.5 + Dart · Flame 2D engine · SQLite (sqflite)
Provider state management · GetIt DI · Clean Architecture
5-language i18n · Offline-first · Android & iOS

Open source · Built for the love of reading and game design
Fernando Pinto Villarroel